

OPERATING SYSTEMS CODES

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

typedef struct {
    int pid;
    int arrival;
    int burst;
    int priority;

    int completion;
    int waiting;
    int turnaround;
    int response;

    int remaining;
    int started;
} Process;

/* =====
FCFS
===== */

void fcfs(Process p[], int n)
{
    int time = 0;
    float avg_wait = 0;
    float avg_turnaround = 0;

    printf("\n===== FCFS =====\n");

    for (int i = 0; i < n; i++)
    {
        if (time < p[i].arrival)
            time = p[i].arrival;

        p[i].response = time - p[i].arrival;

        time += p[i].burst;
    }
}
```

```

    p[i].completion = time;

    p[i].turnaround =
        p[i].completion - p[i].arrival;

    p[i].waiting =
        p[i].turnaround - p[i].burst;

    avg_wait += p[i].waiting;
    avg_turnaround += p[i].turnaround;
}

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

for (int i = 0; i < n; i++)
{
    printf(
        "P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid,
        p[i].arrival,
        p[i].burst,
        p[i].completion,
        p[i].turnaround,
        p[i].waiting,
        p[i].response
    );
}

printf(
    "\nAverage Waiting Time = %.2f\n",
    avg_wait / n
);

printf(
    "\nAverage Turnaround Time = %.2f\n",
    avg_turnaround / n
);
}

/* =====
SJF - NON PREEMPTIVE
===== */

```

```

void sjf(Process p[], int n)
{
    int completed = 0;
    int time = 0;

    int visited[MAX] = {0};

    float avg_wait = 0;
    float avg_turnaround = 0;

    printf("\n===== SJF =====\n");

    while (completed < n)
    {
        int index = -1;
        int shortest = 999999;

        for (int i = 0; i < n; i++)
        {
            if (!visited[i] &&
                p[i].arrival <= time &&
                p[i].burst < shortest)
            {
                shortest = p[i].burst;
                index = i;
            }
        }

        /*
         * CPU idle
         */

        if (index == -1)
        {
            time++;
            continue;
        }

        p[index].response =
            time - p[index].arrival;

        time += p[index].burst;

        p[index].completion = time;
    }
}

```

```

    p[index].turnaround =
        p[index].completion -
        p[index].arrival;

    p[index].waiting =
        p[index].turnaround -
        p[index].burst;

    visited[index] = 1;

    completed++;

    avg_wait += p[index].waiting;

    avg_turnaround += p[index].turnaround;
}

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

for (int i = 0; i < n; i++)
{
    printf(
        "P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid,
        p[i].arrival,
        p[i].burst,
        p[i].completion,
        p[i].turnaround,
        p[i].waiting,
        p[i].response
    );
}

printf(
    "\nAverage Waiting Time = %.2f\n",
    avg_wait / n
);

printf(
    "Average Turnaround Time = %.2f\n",
    avg_turnaround / n
);
}

```

```

/* =====
PRIORITY - NON PREEMPTIVE
Lower number = Higher priority
===== */

void priority_scheduling(Process p[], int n)
{
    int completed = 0;
    int time = 0;

    int visited[MAX] = {0};

    float avg_wait = 0;
    float avg_turnaround = 0;

    printf("\n===== PRIORITY =====\n");

    while (completed < n)
    {
        int index = -1;
        int best_priority = 999999;

        for (int i = 0; i < n; i++)
        {
            if (!visited[i] &&
                p[i].arrival <= time &&
                p[i].priority < best_priority)
            {
                best_priority = p[i].priority;
                index = i;
            }
        }

        if (index == -1)
        {
            time++;
            continue;
        }

        p[index].response =
            time - p[index].arrival;
    }
}

```

```

time += p[index].burst;

p[index].completion = time;

p[index].turnaround =
    p[index].completion -
    p[index].arrival;

p[index].waiting =
    p[index].turnaround -
    p[index].burst;

visited[index] = 1;

completed++;

avg_wait += p[index].waiting;

avg_turnaround += p[index].turnaround;
}

printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");

for (int i = 0; i < n; i++)
{
    printf(
        "P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid,
        p[i].arrival,
        p[i].burst,
        p[i].priority,
        p[i].completion,
        p[i].turnaround,
        p[i].waiting,
        p[i].response
    );
}

printf(
    "\nAverage Waiting Time = %.2f\n",
    avg_wait / n
);

printf(

```

```

        "Average Turnaround Time = %.2f\n",
        avg_turnaround / n
    );
}

/* =====
ROUND ROBIN
===== */

void round_robin(Process p[], int n, int quantum)
{
    int time = 0;
    int completed = 0;

    int queue[MAX * 10];
    int front = 0;
    int rear = 0;

    int added[MAX] = {0};

    float avg_wait = 0;
    float avg_turnaround = 0;

    for (int i = 0; i < n; i++)
    {
        p[i].remaining = p[i].burst;
        p[i].started = 0;
    }

    printf("\n===== ROUND ROBIN =====\n");

    /*
    * Start with processes that have
    * already arrived.
    */

    for (int i = 0; i < n; i++)
    {
        if (p[i].arrival <= time &&
            !added[i])
        {
            queue[rear++] = i;
            added[i] = 1;

```

```
}  
}
```

```
while (completed < n)
```

```
{
```

```
    /*
```

```
    * If queue is empty,
```

```
    * advance CPU time.
```

```
    */
```

```
    if (front == rear)
```

```
    {
```

```
        time++;
```

```
        for (int i = 0; i < n; i++)
```

```
        {
```

```
            if (p[i].arrival <= time &&  
                !added[i])
```

```
            {
```

```
                queue[rear++] = i;
```

```
                added[i] = 1;
```

```
            }
```

```
        }
```

```
        continue;
```

```
    }
```

```
    int index = queue[front++];
```

```
    /*
```

```
    * First response.
```

```
    */
```

```
    if (!p[index].started)
```

```
    {
```

```
        p[index].response =
```

```
            time - p[index].arrival;
```

```
        p[index].started = 1;
```

```
    }
```

```
    int execution_time;
```

```
    if (p[index].remaining < quantum)
```

```

        execution_time = p[index].remaining;
    else
        execution_time = quantum;

    time += execution_time;

    p[index].remaining -= execution_time;

    /*
     * Add newly arrived processes.
     */

    for (int i = 0; i < n; i++)
    {
        if (p[i].arrival <= time &&
            !added[i])
        {
            queue[rear++] = i;
            added[i] = 1;
        }
    }

    /*
     * Process finished.
     */

    if (p[index].remaining == 0)
    {
        p[index].completion = time;

        p[index].turnaround =
            p[index].completion -
            p[index].arrival;

        p[index].waiting =
            p[index].turnaround -
            p[index].burst;

        avg_wait += p[index].waiting;

        avg_turnaround += p[index].turnaround;

        completed++;
    }

```

```

        else
        {
            /*
             * Put process back into queue.
             */

            queue[rear++] = index;
        }
    }

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

    for (int i = 0; i < n; i++)
    {
        printf(
            "P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid,
            p[i].arrival,
            p[i].burst,
            p[i].completion,
            p[i].turnaround,
            p[i].waiting,
            p[i].response
        );
    }

    printf(
        "\nAverage Waiting Time = %.2f\n",
        avg_wait / n
    );

    printf(
        "\nAverage Turnaround Time = %.2f\n",
        avg_turnaround / n
    );
}

/* =====
MAIN
===== */

int main()
{

```

```

int n;

Process original[MAX];

printf("CPU Scheduling Simulator\n");

printf(
    "\nEnter number of processes: "
);

scanf("%d", &n);

for (int i = 0; i < n; i++)
{
    original[i].pid = i + 1;

    printf(
        "\nProcess P%d\n",
        i + 1
    );

    printf("Arrival Time: ");
    scanf(
        "%d",
        &original[i].arrival
    );

    printf("Burst Time: ");
    scanf(
        "%d",
        &original[i].burst
    );

    printf("Priority: ");
    scanf(
        "%d",
        &original[i].priority
    );
}

int choice;

do
{

```

```
printf("\n");
printf("=====\n");
printf("CPU Scheduling Algorithms\n");
printf("=====\n");
printf("1. FCFS\n");
printf("2. SJF\n");
printf("3. Priority\n");
printf("4. Round Robin\n");
printf("5. Exit\n");
```

```
printf(
    "Enter choice: "
);
```

```
scanf(
    "%d",
    &choice
);
```

```
Process temp[MAX];
```

```
for (int i = 0; i < n; i++)
    temp[i] = original[i];
```

```
switch (choice)
{
    case 1:
        fcfs(temp, n);
        break;

    case 2:
        sjf(temp, n);
        break;

    case 3:
        priority_scheduling(temp, n);
        break;

    case 4:
    {
        int quantum;

        printf(
            "Enter time quantum: "
```

```

        );

        scanf(
            "%d",
            &quantum
        );

        round_robin(
            temp,
            n,
            quantum
        );

        break;
    }

    case 5:
        printf(
            "Exiting...\n"
        );
        break;

    default:
        printf(
            "Invalid choice.\n"
        );
    }

} while (choice != 5);

return 0;
}

```

Process synchronization codes

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define NUM_ITEMS 10
#define NUM_PHILOSOPHERS 5

```

```
#define NUM_READERS 3
#define NUM_WRITERS 2
```

```
/* =====
   PRODUCER - CONSUMER
   ===== */
```

```
int buffer[BUFFER_SIZE];
```

```
int in = 0;
int out = 0;
```

```
sem_t empty_slots;
sem_t full_slots;
```

```
pthread_mutex_t buffer_mutex;
```

```
/* -----
   PRODUCER
   ----- */
```

```
void *producer(void *arg)
{
    (void)arg;

    for (int item = 1;
         item <= NUM_ITEMS;
         item++)
    {
        /*
         * Wait for an empty slot.
         */

        sem_wait(
            &empty_slots
        );

        /*
         * Lock shared buffer.
         */
```

```

pthread_mutex_lock(
    &buffer_mutex
);

buffer[in] = item;

printf(
    "Producer produced: %d\n",
    item
);

in =
    (in + 1) %
    BUFFER_SIZE;

pthread_mutex_unlock(
    &buffer_mutex
);

/*
 * Signal that an item exists.
 */

sem_post(
    &full_slots
);

sleep(1);
}

return NULL;
}

/* -----
CONSUMER
----- */

void *consumer(void *arg)

```

```

{
    (void)arg;

    for (int i = 0;
        i < NUM_ITEMS;
        i++)
    {
        /*
         * Wait for available item.
         */

        sem_wait(
            &full_slots
        );

        pthread_mutex_lock(
            &buffer_mutex
        );

        int item = buffer[out];

        printf(
            "Consumer consumed: %d\n",
            item
        );

        out =
            (out + 1) %
            BUFFER_SIZE;

        pthread_mutex_unlock(
            &buffer_mutex
        );

        /*
         * Signal empty slot.
         */

        sem_post(

```

```

        &empty_slots
    );

    sleep(2);
}

return NULL;
}

/* -----
   PRODUCER-CONSUMER DEMO
   ----- */

void producer_consumer()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    sem_init(
        &empty_slots,
        0,
        BUFFER_SIZE
    );

    sem_init(
        &full_slots,
        0,
        0
    );

    pthread_mutex_init(
        &buffer_mutex,
        NULL
    );

    pthread_create(
        &producer_thread,
        NULL,
        producer,

```

```

        NULL
    );

    pthread_create(
        &consumer_thread,
        NULL,
        consumer,
        NULL
    );

    pthread_join(
        producer_thread,
        NULL
    );

    pthread_join(
        consumer_thread,
        NULL
    );

    sem_destroy(
        &empty_slots
    );

    sem_destroy(
        &full_slots
    );

    pthread_mutex_destroy(
        &buffer_mutex
    );
}

/* =====
READERS - WRITERS
===== */

int shared_data = 0;

```

```
int reader_count = 0;
```

```
pthread_mutex_t reader_mutex;
```

```
sem_t writer_semaphore;
```

```
/* -----  
  READER  
  ----- */
```

```
void *reader(void *arg)
```

```
{  
    int id =  
        *(int *)arg;
```

```
/*  
 * Protect reader_count.  
 */
```

```
pthread_mutex_lock(  
    &reader_mutex  
);
```

```
reader_count++;
```

```
/*  
 * First reader blocks writers.  
 */
```

```
if (reader_count == 1)  
{  
    sem_wait(  
        &writer_semaphore  
    );  
}
```

```
pthread_mutex_unlock(  
    &reader_mutex  
);
```

```
/*
 * Read shared data.
 */

printf(
    "Reader %d reads value: %d\n",
    id,
    shared_data
);

sleep(1);

/*
 * Reader finished.
 */

pthread_mutex_lock(
    &reader_mutex
);

reader_count--;

/*
 * Last reader allows writer.
 */

if (reader_count == 0)
{
    sem_post(
        &writer_semaphore
    );
}

pthread_mutex_unlock(
    &reader_mutex
);
```

```

    return NULL;
}

/* -----
WRITER
----- */

void *writer(void *arg)
{
    int id =
        *(int *)arg;

    /*
     * Only one writer at a time.
     */

    sem_wait(
        &writer_semaphore
    );

    shared_data++;

    printf(
        "Writer %d writes value: %d\n",
        id,
        shared_data
    );

    sleep(1);

    sem_post(
        &writer_semaphore
    );

    return NULL;
}

```

```
/* -----  
READERS-WRITERS DEMO  
----- */
```

```
void readers_writers()  
{  
    pthread_t readers[NUM_READERS];  
  
    pthread_t writers[NUM_WRITERS];  
  
    int reader_ids[NUM_READERS];  
  
    int writer_ids[NUM_WRITERS];
```

```
    pthread_mutex_init(  
        &reader_mutex,  
        NULL  
    );
```

```
    sem_init(  
        &writer_semaphore,  
        0,  
        1  
    );
```

```
/*  
 * Create readers.  
 */
```

```
for (int i = 0;  
     i < NUM_READERS;  
     i++)  
{  
    reader_ids[i] = i + 1;
```

```
    pthread_create(  
        &readers[i],  
        NULL,  
        reader,
```

```

        &reader_ids[i]
    );
}

/*
 * Create writers.
 */

for (int i = 0;
     i < NUM_WRITERS;
     i++)
{
    writer_ids[i] = i + 1;

    pthread_create(
        &writers[i],
        NULL,
        writer,
        &writer_ids[i]
    );
}

/*
 * Wait for readers.
 */

for (int i = 0;
     i < NUM_READERS;
     i++)
{
    pthread_join(
        readers[i],
        NULL
    );
}

/*
 * Wait for writers.
 */

for (int i = 0;

```

```

        i < NUM_WRITERS;
        i++)
    {
        pthread_join(
            writers[i],
            NULL
        );
    }

    pthread_mutex_destroy(
        &reader_mutex
    );

    sem_destroy(
        &writer_semaphore
    );
}

/* =====
DINING PHILOSOPHERS
===== */

pthread_mutex_t forks[
    NUM_PHILOSOPHERS
];

/* -----
PHILOSOPHER
----- */

void *philosopher(void *arg)
{
    int id =
        *(int *)arg;

    int left =
        id;

    int right =

```

```

        (id + 1) %
        NUM_PHILOSOPHERS;

printf(
    "Philosopher %d is thinking.\n",
    id
);

sleep(1);

/*
 * Avoid deadlock by changing
 * fork acquisition order for
 * the last philosopher.
 */

if (id == NUM_PHILOSOPHERS - 1)
{
    pthread_mutex_lock(
        &forks[right]
    );

    pthread_mutex_lock(
        &forks[left]
    );
}
else
{
    pthread_mutex_lock(
        &forks[left]
    );

    pthread_mutex_lock(
        &forks[right]
    );
}

printf(
    "Philosopher %d is eating.\n",
    id

```

```

);

sleep(2);

pthread_mutex_unlock(
    &forks[left]
);

pthread_mutex_unlock(
    &forks[right]
);

printf(
    "Philosopher %d finished eating.\n",
    id
);

return NULL;
}

/* -----
DINING PHILOSOPHERS DEMO
----- */

void dining_philosophers()
{
    pthread_t philosophers[
        NUM_PHILOSOPHERS
    ];

    int ids[
        NUM_PHILOSOPHERS
    ];

    /*
     * Initialize forks.
     */

```

```
for (int i = 0;
     i < NUM_PHILOSOPHERS;
     i++)
{
    pthread_mutex_init(
        &forks[i],
        NULL
    );
}
```

```
/*
 * Create philosophers.
 */
```

```
for (int i = 0;
     i < NUM_PHILOSOPHERS;
     i++)
{
    ids[i] = i;

    pthread_create(
        &philosophers[i],
        NULL,
        philosopher,
        &ids[i]
    );
}
```

```
/*
 * Wait for philosophers.
 */
```

```
for (int i = 0;
     i < NUM_PHILOSOPHERS;
     i++)
{
    pthread_join(
        philosophers[i],
        NULL
    );
}
```

```

/*
 * Destroy mutexes.
 */

for (int i = 0;
     i < NUM_PHILOSOPHERS;
     i++)
{
    pthread_mutex_destroy(
        &forks[i]
    );
}

/* =====
MAIN MENU
===== */

int main()
{
    int choice;

    do
    {
        printf("\n");
        printf("===== \n");
        printf("  PROCESS SYNCHRONIZATION LAB\n");
        printf("===== \n");

        printf(
            "1. Producer-Consumer\n"
        );

        printf(
            "2. Readers-Writers\n"
        );

        printf(
            "3. Dining Philosophers\n"
        );
    }

```

```
printf(  
    "4. Exit\n"  
);
```

```
printf(  
    "\nEnter choice: "  
);
```

```
scanf(  
    "%d",  
    &choice  
);
```

```
switch (choice)  
{  
    case 1:  
  
        printf(  
            "\n--- Producer Consumer ---\n"  
        );  
  
        producer_consumer();  
  
        break;
```

```
    case 2:  
  
        printf(  
            "\n--- Readers Writers ---\n"  
        );  
  
        readers_writers();  
  
        break;
```

```
    case 3:  
  
        printf(  
            "\n--- Dining Philosophers ---\n"  
        );
```

```
dining_philosophers();

break;

case 4:

    printf(
        "Exiting...\n"
    );

    break;

default:

    printf(
        "Invalid choice.\n"
    );
}

} while (choice != 4);

return 0;
}
```
